Guía de Arquitectura de Sistemas y Escalabilidad

Estrategias de Almacenamiento Multimedia, Migración Self-Hosted, Concurrencia y Pasarelas de Pago

1. Almacenamiento de Fotos e Imágenes: ¿Por qué NUNCA en la Base de Datos?

Guardar imágenes como binarios (`BYTEA` , BLOBs o Base64) directamente en una tabla de PostgreSQL es una práctica desaconsejada en producción por cuatro motivos críticos:

Impacto destructivo de guardar imágenes en la base de datos:

- **Contaminación de la memoria RAM (Buffer Pool):**
  PostgreSQL guarda en memoria páginas de datos para responder en microsegundos. Una sola foto de 4 MB expulsa de la RAM el espacio de más de 10.000 filas de órdenes o clientes, forzando a la base de datos a leer de disco mecánico/SSD continuo.
- **Backups inmanejables (Bloat):**
  5.000 órdenes con 3 fotos generan más de 45 GB de datos estáticos. El respaldo diario (`pg_dump`) tardará horas, consumirá terabytes de transferencia y disparará el tamaño de los logs de transacciones (WAL).
- **Saturación de I/O y conexiones:**
  Cuando un cliente consulta su equipo desde su teléfono, la base de datos debe enviar megabytes por su conexión TCP interna, bloqueando hilos de procesamiento.

La Práctica Estándar de la Industria: Object Storage + CDN + URLs en la DB

La solución estándar adoptada por gigantes de la industria consiste en desacoplar por completo el almacenamiento binario:

```
[Técnico / Cliente] | ▼ (1) Sube foto directamente vía Pre-signed URL [Object Storage: Cloudflare R2 / AWS S3 / Supabase Storage] | ▼ (2) Retorna la URL pública optimizada (ej: WebP) [Frontend RepairIT] | ▼ (3) Guarda únicamente la URL en la base de datos (campo VARCHAR) [PostgreSQL: orders.photos = '["https://cdn.repairit.com/photos/a54-front.webp"]'] | ▼ (4) Al consultar portal de seguimiento [Cliente Final] ◄── (5) Descarga en <20ms desde servidores perimetrales ── [Cloudflare CDN]
```

Ventajas: 1) **Costo casi nulo:** Almacenar 100 GB en Cloudflare R2 cuesta \$1.5 USD al mes y **no cobra por tráfico de descarga (egress free)**. 2) **Base de datos ligera:** La tabla de órdenes solo almacena textos pequeños de unos pocos bytes. 3) **Velocidad extrema:** Las fotos se sirven desde servidores CDN perimetrales a menos de 20 ms del cliente.

2. Independencia de Supabase & Self-Hosting en un VPS

Sí, podés dejar de depender de Supabase Cloud en cualquier momento. Supabase es 100% código abierto y está compuesto por herramientas estándares de software libre.

Opción A: Supabase Self-Hosted en tu propio servidor VPS (Recomendada)

Podés desplegar la suite completa de Supabase en un servidor VPS de proveedores como **Hetzner, DigitalOcean o AWS** mediante su `docker-compose.yml` oficial o a través de plataformas como **Coolify**:

Componente	Tecnología	Función en RepairIT
Base de Datos	PostgreSQL 15/16	Motor relacional central con extensiones completas.
Motor API	PostgREST (Haskell)	Genera automáticamente endpoints REST a partir de tablas y vistas.
Autenticación	GoTrue (Go)	Microservicio de registro, login, contraseñas bcrypt y JWT.
Almacenamiento	Storage API (NodeJS)	Gestión de archivos binarios hacia disco local o buckets S3/MinIO.
Gateway & Proxy	Kong API Gateway	Terminación SSL, control de cabeceras y enrutamiento interno.
Administración	Supabase Studio	Panel visual web y editor SQL idéntico a Supabase Cloud.

> **Costo Real:**
> Un servidor VPS en Hetzner (ej. CPX31 de 4 vCPUs y 8 GB de RAM) cuesta aproximadamente **€14 al mes** y tiene potencia suficiente para gestionar más de 200 talleres concurrentes sin pagar tarifas por usuario.

Opción B: PostgreSQL Nativo + Backend Tradicional (Node.js / Fastify / Go)

Si preferís no usar la arquitectura de PostgREST:

- Levantar una base de datos PostgreSQL estándar afinando parámetros de memoria (`shared_buffers = 25% RAM` , `work_mem = 16MB`).
- Crear un backend propio en **Node.js (Fastify/NestJS)** o **Go** con un ORM/Query Builder moderno (**Drizzle ORM** o **Prisma**).
- Reemplazar en el frontend el cliente de `@supabase/supabase-js` por un cliente HTTP estándar como `axios` hacia tu dominio API.

3. Arquitectura para Alta Concurrencia (Miles de Usuarios Simultáneos)

Cuando el sistema crece y cientos de talleres operan al mismo tiempo, los cuellos de botella ocurren en lugares predecibles. A continuación, el plano de arquitectura para evitar caídas:

A. Connection Pooling con PgBouncer / Supavisor (Indispensable)

PostgreSQL crea un proceso por cada cliente conectado (~10 MB de RAM por conexión). A las 250 conexiones simultáneas el servidor colapsa por cambio de contexto y falta de memoria.

La solución: Interpolar **PgBouncer** o **Supavisor** entre los clientes y PostgreSQL. Permite que **5.000 clientes concurrentes** compartan un pool de solo **30 a 50 conexiones físicas reales** a la base de datos.

B. Balanceo de Carga Web (Nginx / Cloudflare / Traefik)

- **Cloudflare (Capa 1):** Resuelve DNS por Anycast, filtra tráfico malicioso (WAF), mitiga ataques DDoS y sirve en caché todo el frontend estático de Vite (JS, CSS, HTML). El 80% de las peticiones a la plataforma ni siquiera tocan tu servidor.
- **Nginx / Traefik (Capa 2):** Distribuye el tráfico dinámico de la API en modo *Round-Robin* o *Least Connections* entre múltiples instancias de la aplicación.

C. Base de Datos de Alta Disponibilidad: Maestro y Réplicas de Lectura

En un software de talleres, **el 85% al 90% del tráfico son lecturas** (`SELECT` para ver órdenes, historial, caja, clientes) y solo el 10-15% son escrituras (`INSERT` o `UPDATE`).

```
[Tráfico de Clientes y Técnicos] | ▼ [Balanceador de Carga / Proxy] |
┌──────────────────────┴──────────────────────┐ ▼ (Escrituras: INSERT, UPDATE, DELETE) ▼ (Lecturas: SELECT)
[Nodo Maestro (Primary)] [Nodos Réplica (Read Only)] | (Sincronización en tiempo real) └──────
Streaming Replication ──────────────────► [Réplica 1] ► [Réplica 2]
```

- **Failover Automático con Patroni:** Si el nodo Maestro se apaga o falla el hardware, el clúster con **Patroni y etcd** promueve automáticamente una de las réplicas a nuevo Maestro en menos de 5 segundos sin intervención humana (*Zero Downtime Failover*).

4. Backups en Tiempo Real: Recuperación Punto en el Tiempo (PITR)

Un backup nocturno tradicional (`pg_dump`) no es suficiente: si el disco falla a las 23:50 hs, se pierden todas las órdenes registradas durante el día.

La solución moderna: WAL-G o **pgBackRest**.

- Se toma una copia base semanal.
- Cada transacción escrita en el registro de escritura anticipada (WAL) se comprime y se envía inmediatamente a un bucket seguro en la nube (ej. AWS S3 en otra región).
- **Recuperación quirúrgica:** Si un operario borra una tabla por error a las 14:15:30 hs, la base de datos se puede restaurar con exactitud de segundos al estado de las **14:15:29 hs**.

5. Integración de Pasarelas de Pago (Mercado Pago / Stripe)

Para procesar cobros automáticos de presupuestos en el portal público o cobrar planes de suscripción de talleres:

```
1. Cliente presiona "Aprobar y Pagar Presupuesto" en Portal de Seguimiento. 2. Backend genera la Preferencia de Pago en Mercado Pago / Stripe con 'external_reference'. 3. Cliente abona con Tarjeta, Dinero en Cuenta o Transferencia. 4. Pasarela envía un WEBHOOK asíncrono (HTTP POST) al Backend. 5. Backend valida firma criptográfica HMAC-SHA256 del webhook. 6. Se ejecuta transacción atómica en PostgreSQL: - Registra el cobro en la tabla 'sales'. - Actualiza la orden a 'presupuesto_aprobado' o 'entregado'. - Descuenta stock en 'inventory'. 7. Backend responde HTTP 200 OK a la pasarela.
```

Principio de Idempotencia:

Las pasarelas de pago reintentan enviar el webhook varias veces si la conexión titubea. En la base de datos se debe agregar un índice único `payment_id` para verificar si ese cobro ya se registró previamente y evitar duplicar cobros o descontar stock dos veces.

6. Hoja de Ruta de Infraestructura Recomendada por Etapa

Etapa del Proyecto	Capacidad de Talleres	Infraestructura Sugerida	Costo Aprox. Mensual
Fase 1: Lanzamiento & MVP	1 a 100 talleres	Supabase Pro Managed + Vercel / Netlify Frontend + Cloudflare DNS	\$25 - \$40 USD
Fase 2: Consolidación	100 a 1.000 talleres	VPS Hetzner / DigitalOcean (Coolify / Docker: Postgres + PgBouncer + Cloudflare R2 Storage)	\$35 - \$70 USD
Fase 3: Escala Empresarial	+1.000 talleres	Clúster Alta Disponibilidad: 2 Nodos Web + Postgres Maestro + 2 Read Replicas (Patroni) + Backups PITR en S3	\$180 - \$350 USD